

Comunicación I2C entre 4 Arduinos

Reporte de práctica — Sistemas Programables

Ximena Armenta Davila

1. Objetivo

Entender cómo funciona el bus de comunicación I2C conectando un Arduino maestro con tres Arduinos esclavos, cada uno con una dirección distinta (0x08, 0x09 y 0x0A). Se busca aprender a armar el bus, asignar direcciones y programar el envío y la recepción de datos entre las placas.

2. Cómo funciona

El bus I2C es una forma de comunicar varios dispositivos usando solamente dos cables de datos: SDA (datos) y SCL (reloj), además de una tierra común (GND). Todos los Arduinos se conectan a las mismas dos líneas, y lo que los distingue es su dirección, un número único que cada uno tiene asignado.

En esta práctica hay un Arduino maestro y tres Arduinos esclavos:

- Esclavo 1 (dirección 0x08): recibe una orden del maestro y con ella enciende o apaga un LED.
- Esclavo 2 (dirección 0x09): recibe un ángulo (de 0° a 180°) y mueve un servomotor a esa posición.
- Esclavo 3 (dirección 0x0A): lee un potenciómetro y envía su valor al maestro cuando este se lo pide.

El maestro es quien controla toda la comunicación: decide con quién habla y cuándo. Los esclavos nunca hablan por su cuenta, solo responden cuando el maestro los llama. El programa del maestro hace dos cosas de forma repetida:

1. Cada 500 milisegundos, le pide al esclavo 3 el valor del potenciómetro, lo convierte a un ángulo de 0° a 180° con la función `map()`, y se lo envía al esclavo 2 para mover el servomotor.
2. Todo el tiempo revisa si el usuario escribió algo en el Monitor serie: si escribe 1 o 0, le manda esa orden al esclavo 1 para encender o apagar el LED.

Para no quedarse "congelado" esperando, el maestro usa `millis()` en lugar de `delay()`, así puede atender el Monitor serie y consultar el potenciómetro casi al mismo tiempo. También revisa el resultado de `endTransmission()`: si no es 0, quiere decir que ese esclavo no respondió (NACK), y el maestro lo avisa en el Monitor serie en vez de trabarse.

Diagrama de conexión

Las cuatro placas comparten las líneas SDA, SCL y GND. El LED con su resistencia y el servomotor están conectados al lado de los esclavos 1 y 2, y el potenciómetro al esclavo 3.

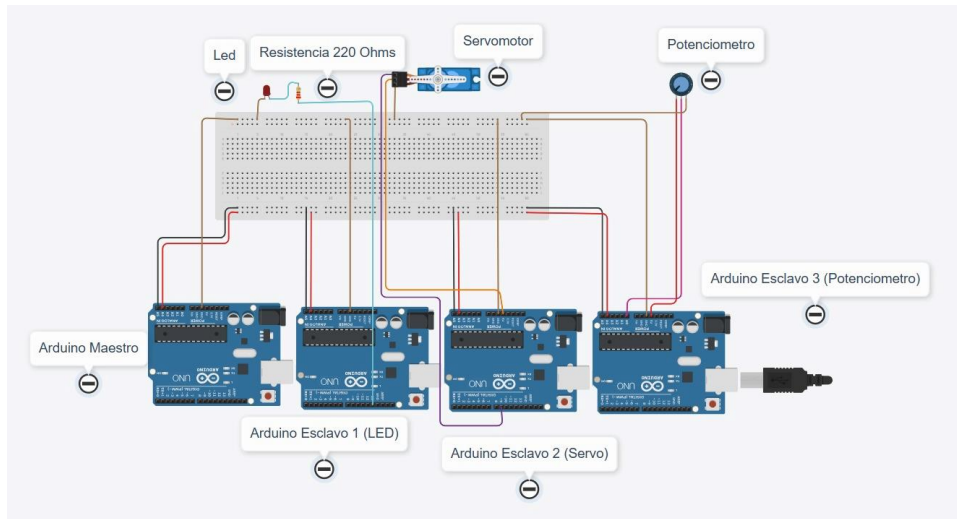


Figura 1. Diagrama del circuito con el Arduino maestro y los tres esclavos.

Diferencias entre el R3 y el R4 WiFi

El código base se escribió pensando en un Arduino Uno R3, pero las placas usadas en la práctica son R4 WiFi, así que hubo que hacer algunos ajustes:

- El puerto serie del R4 es USB nativo, así que al maestro se le agregó una espera al inicio para no perder los primeros mensajes.
- La resistencia del LED cambia de 220 Ω a 470 Ω , porque los pines del R4 soportan menos corriente que los del R3.
- En una conexión física real (no en simulador) se necesitan resistencias pull-up de 4.7 k Ω en SDA y SCL, ya que el R4 no las trae tan fuertes.
- Los GND de las cuatro placas se unen, pero no los 5V: cada placa se alimenta de su propio cable USB.

3. Material utilizado

- 4 Arduino Uno R4 WiFi
- 1 protoboard
- 1 LED
- 1 resistencia (220 Ω en el código original, 470 Ω en el R4)
- 1 micro servomotor
- 1 potenciómetro
- Cables de conexión
- Librerías Wire y Servo, y el Monitor serie de Arduino

4. Resultados

Se armó el circuito de manera física con las cuatro placas Arduino R4 WiFi, el protoboard, el LED, el servomotor y el potenciómetro, todos compartiendo las líneas SDA, SCL y GND.

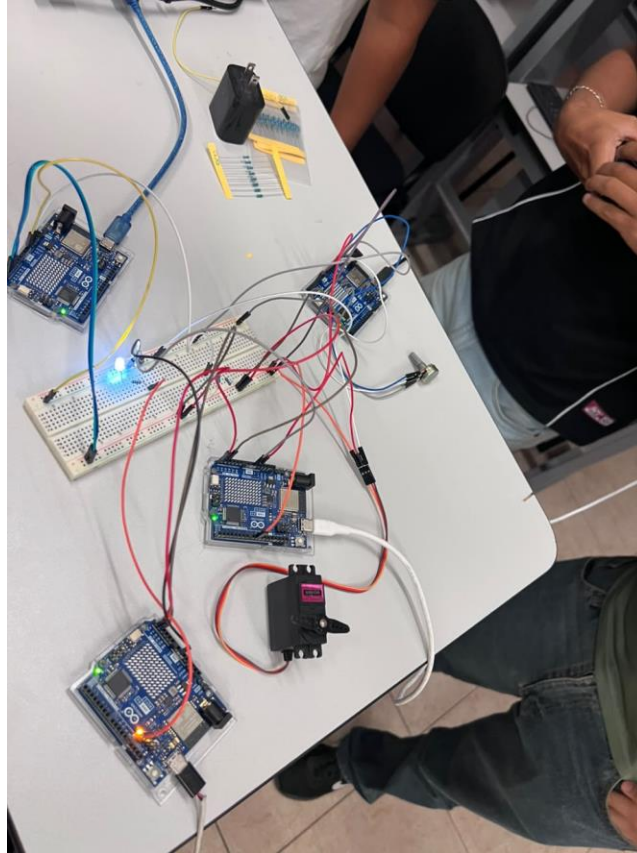


Figura 2. Montaje físico del circuito con las cuatro placas Arduino.

Al cargar los programas y abrir el Monitor serie del maestro, se observó que cada 500 ms llegaba una nueva lectura del potenciómetro, la cual se convertía correctamente a un ángulo entre 0° y 180° y se enviaba al servomotor. También se pudo encender y apagar el LED escribiendo 1 o 0 en el Monitor serie.

```

Monitor en serie

Maestro listo. Escribe 1 (LED encendido) o 0 (LED apagado)
Potenciómetro: 307 -> servo a 54 grados
Potenciómetro: 307 -> servo a 54 grados
Potenciómetro: 307 -> servo a 54 grados
Potenciómetro: 0 -> servo a 0 grados
Potenciómetro: 818 -> servo a 143 grados
Potenciómetro: 818 -> servo a 143 grados
Potenciómetro: 1023 -> servo a 180 grados
  
```

Figura 3. Monitor serie mostrando las lecturas del potenciómetro y el ángulo enviado al servo.

Como se ve en la Figura 3, valores bajos del potenciómetro (por ejemplo 0) mandaron el servo a 0° , valores intermedios (como 307 u 818) lo movieron a ángulos intermedios (54° y 143°), y el valor máximo (1023) lo llevó a 180° . Esto confirma que la conversión con `map()` y el envío por I2C funcionaron de forma correcta, y que el LED y el servo respondieron sin errores de comunicación (no se imprimió ningún mensaje de "no responde").

5. Conclusiones

Con esta práctica pude entender de forma más clara cómo funciona la comunicación I2C entre varios dispositivos. Al armar el bus con el Arduino maestro y los tres esclavos me di cuenta de que, aunque todos comparten las mismas líneas SDA y SCL, lo que realmente los distingue es su dirección, y que basta con darle una dirección diferente a cada uno para poder agregar más dispositivos sin necesitar más pines. También noté la importancia de que el maestro sea quien controla toda la comunicación, ya que si dos esclavos tuvieran la misma dirección los datos se

corromperían, y por eso es clave revisar el resultado de `endTransmission()` para saber si un esclavo no está respondiendo en lugar de que el programa se quede trabado.

Otro aprendizaje importante fue usar `millis()` en lugar de `delay()` en el código del maestro, ya que esto me permitió atender el Monitor serie y al mismo tiempo seguir consultando el potenciómetro sin que el programa se detuviera. Durante el armado físico del circuito también cometí algunos errores típicos, como colocar mal una resistencia en la protoboard, lo cual me ayudó a entender por qué es tan importante revisar bien las conexiones antes de energizar el circuito. Al ver en el Monitor serie que las lecturas del potenciómetro se convertían correctamente en ángulos para el servomotor, y que el LED respondía sin problema a las órdenes enviadas, confirmé que el circuito y los programas funcionaron como se esperaba, y que logré cumplir el objetivo de la práctica.